

RESTAURARTE

Documentação Técnica do Sistema

Programa de Gerenciamento de Restaurante via Terminal (TUI)

Cibely Rodrigues de Pontes

Daniel Camargos da Cunha

Ian Fernandes Guimarães

Miguel Dolabella Miranda

Pedro Henrique Penha Piantino

VESPASIANO

2026

RESTAURARTE

Documentação Técnica do Sistema

Trabalho apresentado à disciplina de Algoritmos e Programação do Curso de Ciência da Computação da Faculdade da Saúde e Ecologia Humana (FASEH), como requisito avaliativo.

VESPASIANO
2026

RESUMO

Esta documentação apresenta, de forma detalhada e sistematizada, a arquitetura técnica, os fluxos de funcionamento e as decisões de projeto do sistema RestaurARTE um programa de gerenciamento de restaurante executado inteiramente em terminal (TUI: Terminal User Interface).

O sistema foi desenvolvido em linguagem Java 21 utilizando o banco de dados SQLite para persistência local de dados, com arquitetura organizada em camadas: interface de usuário, serviços de negócio, modelos de dados e camada de acesso a banco.

São documentados a visão geral do projeto, o design do sistema em classes, a arquitetura do banco de dados com cinco tabelas relacionadas, os fluxos completos de navegação, a especificação técnica de cada classe e método, além dos requisitos funcionais e não funcionais.

A documentação serve como referência para manutenção, extensão e apresentação acadêmica do sistema.

SUMÁRIO

1 Visão Geral do Projeto

- 1.1 Contexto e Tema
- 1.2 Problema Resolvido
- 1.3 Público-Alvo
- 1.4 Tecnologias Utilizadas

2 Estrutura do Projeto e Arquitetura

- 2.1 Organização de Diretórios
- 2.2 Diagrama de Camadas
- 2.3 Descrição Geral das Classes

3 Especificação Técnica das Classes

- 3.1 Main.java
- 3.2 Utils.java
- 3.3 Database.java
- 3.4 Modelos de Dados
- 3.5 Services
- 3.6 Interface do Programa

4 Banco de Dados: Esquema e Relacionamentos

- 4.1 Diagrama Entidade-Relacionamento
- 4.2 Especificação das Tabelas
- 4.3 Relacionamentos e Chaves Estrangeiras

5 Fluxos de Navegação e Casos de Uso

- 5.1 Fluxo Completo do Sistema
- 5.2 Caso de Uso: Login e Cadastro

5.3 Caso de Uso: Realizar Pedido

5.4 Caso de Uso: Avaliações

6 Levantamento de Requisitos

6.1 Requisitos Funcionais

6.2 Requisitos Não Funcionais

7 Considerações Finais

1 Visão Geral do Projeto

1.1 Contexto e Tema

O RestaurARTE é um sistema de gerenciamento de restaurante desenvolvido como trabalho avaliativo da disciplina de Algoritmos e Programação do curso de Ciência da Computação da FASEH. O sistema opera exclusivamente através do terminal do sistema operacional, sem nenhuma interface gráfica, caracterizando o paradigma TUI (Terminal User Interface).

O tema central é a automação do ciclo de atendimento de um restaurante: desde o primeiro contato do cliente com o sistema (login ou cadastro), passando pela seleção de itens do cardápio, formação do pedido e encerramento da conta, até a coleta de avaliação da experiência.

1.2 Problema Resolvido

Restaurantes de pequeno e médio porte frequentemente gerenciam suas operações de forma manual através de anotações em papel, planilhas desconexas ou sistemas genéricos que não se adaptam ao fluxo real do negócio. Os principais problemas enfrentados pelo RestaurARTE são:

- ▲ Ausência de histórico digital de clientes e seus pedidos.
- ▲ Dificuldade em controlar quais pedidos estão em aberto e quais já foram encerrados.
- ▲ Falta de mecanismo estruturado para coleta de avaliações de satisfação.
- ▲ Perda de dados entre sessões por ausência de persistência confiável.
- ▲ Necessidade de login complexo com senha, barreira de entrada desnecessária para este contexto.

O sistema resolve esses problemas com uma solução leve, offline, de fácil operação e com persistência total em banco de dados local SQLite. O acesso é feito apenas pelo nome do cliente, sem a utilização de senhas, e todos os dados são mantidos entre execuções do programa.

1.3 Público-Alvo

O sistema foi concebido para dois perfis de usuário que interagem com o terminal:

- ▲ **Operadores do restaurante**, funcionários que utilizam o sistema para registrar a entrada de clientes, acompanhar pedidos em aberto e encerrar contas.
- ▲ **Clientes do restaurante**, frequentadores que consultam o cardápio, adicionam pratos ao pedido, visualizam o total e deixam avaliações.

1.4 Tecnologias Utilizadas

Tecnologia	Função no Projeto
Java 21	Linguagem de programação principal
SQLite 3	Banco de dados local em único arquivo (data/restaurante.db)
JDBC (java.sql.*)	API padrão do Java para comunicação com bancos de dados

sqlite-jdbc 3.47.1.0	Driver JDBC específico para SQLite, adicionado como dependência no build.gradle via Maven Central
Gradle 9.3	Ferramenta de build, gerenciando as dependências e compila o projeto para um executável
Git / GitHub	Controle de versão do projeto e repositório do código

2 Estrutura do Projeto e Arquitetura

2.1 Organização de Diretórios

O projeto segue a convenção de estrutura de projetos Gradle/Maven para Java:

TEXT0

```
trabalho-algoritimo/
■■■ build.gradle ← Configuração do build e dependências
■■■ settings.gradle ← Nome do projeto (restaurarte)
■■■ gradlew / gradlew.bat ← Wrapper do Gradle (Linux / Windows)
■■■ data/
■ ■■■ restaurarte.db ← Banco SQLite (gerado na 1ª execução)
■■■ src/main/java/me/faseh/restaurarte/
■■■ Main.java ← Ponto de entrada do programa
■■■ TUI.java ← Interface com o usuário (todos os menus)
■■■ Utils.java ← Utilitários: leitura de teclado e formatação
■■■ db/
■ ■■■ Database.java ← Singleton: conexão SQLite + DDL das tabelas
■■■ services/
■■■ Cliente.java ← Modelo: dados do cliente
■■■ ClienteService.java ← CRUD: tabela clientes
■■■ Prato.java ← Modelo: dados do prato/cardápio
■■■ PratoService.java ← CRUD: tabela pratos
■■■ Pedido.java ← Modelo: pedido (agrega pedidos + pedido_pratos)
■■■ PedidoService.java ← CRUD: tabelas pedidos e pedido_pratos
■■■ Avaliacao.java ← Modelo: dados da avaliação
■■■ AvaliacaoService.java ← CRUD: tabela avaliacoes
```

2.2 Diagrama de Camadas

O sistema é organizado em quatro camadas com responsabilidades bem delimitadas, seguindo o padrão *Service Layer* (Camada de Serviços):

CAMADA	CLASSES	RESPONSABILIDADE	REGRA
Interface (TUI)	TUI.java	Exibe menus, lê o teclado, gerencia o estado da sessão.	Nunca acessa o banco diretamente. Nunca faz cálculos de negócio.
Serviços	ClienteService PratoService PedidoService AvaliacaoService	Toda a lógica de negócio e operações SQL.	Nunca imprime no console. Lança RuntimeException em erros.
Modelos	Cliente Prato Pedido Avaliacao	Representam entidades do domínio (dados puros).	Apenas campos, construtor, getters e toString(). Sem SQL.
Banco	Database.java	Gerencia a conexão única com o SQLite. Cria as tabelas no 1º uso.	Padrão Singleton. Não executa queries de negócio.

2.3 Descrição Geral das Classes

Classe	Descrição
Main.java	Ponto de entrada. Instancia o TUI e chama menuPrincipal(). Não contém nenhuma lógica de negócio.
TUI.java	A interface completa do sistema. Contém todos os menus (menuPrincipal, menuEntrar, menuRestaurante, menuCardapio, menuAvaliacoes etc.). Mantém o estado da sessão: clienteAtual e pedidoAtual.
Utils.java	Utilitários estáticos reutilizáveis: scnInt(), scnDouble(), scnString() com validação de entrada, e formatarPreco() para formatação monetária brasileira.
Database.java	Implementa o Padrão Singleton para garantir uma única conexão com o banco SQLite durante toda a execução. Responsável pela criação automática das 5 tabelas via DDL na inicialização.
Cliente.java	Modelo POJO (Plain Old Java Object) do cliente. Campos: id, nome, dataEntrada. Inclui getDataEntradaFormatada() para exibição no TUI.
ClienteService.java	CRUD completo para a tabela clientes: registrarCliente(), listarTodos(), buscarPorId(), buscarPorNome() (com COLLATE NOCASE para busca case-insensitive), remover().
Prato.java	Modelo do item do cardápio. Campos: id, nome, preco, tempoMinutos. O toString() formata a linha de exibição do cardápio com alinhamento de colunas.
PratoService.java	CRUD para a tabela pratos. Na inicialização, verifica se a tabela está vazia e, caso positivo, executa semearCardapioPadrao() com 8 pratos pré-definidos.
Pedido.java	Modelo que agrega dados de duas tabelas: pedidos (cabecalho) e pedido_pratos (lista de itens). Controla o estado: aberto (fim=null) ou fechado (fim preenchido + entregue=true).
PedidoService.java	Gerencia todo o ciclo de vida de pedidos: abrirPedido(), adicionarPrato(), fecharPedido(), marcarEntregue(), listarAbertos(), listarPorCliente(), buscarPorId(). Carrega os itens via tabela pedido_pratos.
Avaliacao.java	Modelo de avaliação. Campos: id, idCliente, nomeCliente, nota (0-5), comentario, data. O método getEstrelasVisuais() converte a nota inteira em representação com caracteres ★■.
AvaliacaoService.java	INSERT e SELECT para a tabela avaliacoes. Inclui calcularMedia() que usa AVG(nota) do SQL para retornar a média como OptionalDouble.

3 Especificação Técnica das Classes

3.1 Main.java

Classe responsável exclusivamente pelo ponto de entrada do programa. Instancia o objeto TUI e delega o controle do fluxo a ele:

JAVA

```
public class Main {  
    public static void main(String[] args) {  
        TUI restaurARTE = new TUI();  
        restaurARTE.menuPrincipal();  
    }  
}
```

A separação entre Main e TUI é intencional: mantém a classe de inicialização limpa e permite que o TUI seja instanciado e testado de forma independente, se necessário.

3.2 Utils.java

Classe utilitária com métodos estáticos. Todos são chamados como `Utils.scnInt(sc)` sem necessidade de instanciação. Contém três grupos de funcionalidades:

Constantes visuais

JAVA

```
public static final String equal = "=".repeat(70);  
public static final String dash = "-".repeat(70);
```

Utilizadas em todo o TUI para demarcar seções dos menus no terminal, garantindo uma aparência consistente em toda a interface.

Leitura segura de teclado

O método `scnInt(Scanner)` evita o bug clássico do Java de usar `nextInt()` seguido de `nextLine()`, que resulta em leitura de linha vazia. A solução adotada é sempre ler a linha completa com `nextLine()` e converter com `Integer.parseInt()`:

JAVA

```
public static int scnInt(Scanner scanner) {  
    while (true) {  
        try {  
            String input = scanner.nextLine().trim();  
            if (input.isEmpty()) { /* pede novamente */ continue; }  
            return Integer.parseInt(input);  
        } catch (NumberFormatException e) {  
            System.out.print("Entrada invalida. Digite um numero: ");  
        }  
    }  
}
```

O método `scnDouble()` adicionalmente substitui vírgula por ponto antes de parsear, aceitando tanto 32,50 quanto 32.50 — comportamento essencial para o contexto brasileiro.

Formatação monetária

JAVA

```
public static String formatarPreco(double preco) {  
    return String.format("R$ %.2f", preco).replace(".", ",");  
}  
// Exemplo: formatarPreco(32.5) → "R$ 32,50"
```

3.3 Database.java — O Padrão Singleton

Esta é a classe mais crítica da infraestrutura. Implementa o padrão de projeto **Singleton**, garantindo que exista uma única instância do objeto e, consequentemente, uma única conexão com o banco SQLite durante toda a execução do programa.

Por que Singleton?

Cada chamada a `DriverManager.getConnection()` abre uma nova conexão com o banco. Sem o Singleton, cada um dos quatro services abriria sua própria conexão — criando concorrência desnecessária e possíveis inconsistências. Com o Singleton, todos compartilham a mesma conexão:

JAVA

```
public class Database {
    private static Database instance; // única instância
    private Connection conn; // única conexão

    private Database() { // construtor PRIVADO
        Files.createDirectories(Paths.get("data"));
        conn = DriverManager.getConnection("jdbc:sqlite:data/restaurarte.db");
        conn.createStatement().execute("PRAGMA foreign_keys = ON");
        criarEsquema();
    }

    public static Database getInstance() {
        if (instance == null) instance = new Database();
        return instance;
    }

    public Connection getConn() { return conn; }
}
```

O construtor privado impede que qualquer outra classe execute `new Database()`. O único ponto de acesso é `Database.getInstance()`, que cria a instância na primeira chamada e retorna a mesma instância em todas as chamadas subsequentes.

PRAGMA foreign_keys = ON

O SQLite, por questões de compatibilidade histórica, desativa a verificação de chaves estrangeiras por padrão. Sem esta instrução PRAGMA, seria possível inserir um pedido com um `id_cliente` que não existe na tabela clientes, corrompendo a integridade referencial dos dados. A ativação é feita uma única vez na inicialização do banco.

criarEsquema() — DDL automático

O método executa cinco instruções `CREATE TABLE IF NOT EXISTS`, o que significa que na primeira execução do programa o banco é criado do zero, e nas execuções seguintes as instruções são simplesmente ignoradas — sem nenhuma perda de dados e sem necessidade de migrations manuais.

3.4 Modelos de Dados

Os modelos são classes POJO (Plain Old Java Objects) — apenas carregam dados. Não acessam banco, não imprimem no console. São construídos pelos services após consultar o banco e passados ao TUI para exibição.

Cliente.java

Campo	Tipo Java	Descrição
id	int	Chave primária gerada pelo banco (AUTOINCREMENT).
nome	String	Nome do cliente. Busca case-insensitive via COLLATE NOCASE.
dataEntrada	LocalDate	Data de cadastro. Armazenada como TEXT ISO no banco.

Prato.java

Campo	Tipo Java	Descrição
id	int	Chave primária.
nome	String	Nome do prato. Ex: "Frango Grelhado".
preco	double	Preço em reais. Armazenado como REAL no SQLite.
tempoMinutos	int	Tempo estimado de preparo em minutos.

Pedido.java

Classe de maior complexidade entre os modelos, pois agrega dados de duas tabelas:

Campo	Tipo Java	Descrição/Observação
id	int	PK — de pedidos.
idCliente	int	FK → clientes(id).
inicio	LocalDateTime	Hora de abertura. TEXT ISO no banco (inclui nanosegundos).
fim	LocalDateTime	Hora de encerramento. NULL no banco = pedido em aberto.
entregue	boolean	Mapeado para INTEGER (0/1) no SQLite.
idsPratos	List	Carregado via tabela pedido_pratos. Não existe como coluna em pedidos.

Avaliacao.java

Campo	Tipo Java	Descrição/Observação
id	int	PK.
idCliente	int	FK → clientes(id).
nomeCliente	String	Desnormalizado: guardado na tabela para exibição mesmo se o cliente for removido.

nota	int	Valor de 0 a 5. Validado no TUI antes de chamar o service.
comentario	String	Opcional. Armazenado como TEXT DEFAULT ".
data	LocalDateTime	Momento exato da avaliação.

3.5 Services - Camada de Acesso a Dados

Os quatro services compartilham o mesmo padrão estrutural: obtêm a conexão do Singleton no construtor e expõem métodos CRUD que executam queries via PreparedStatement (para queries com parâmetros externos, prevenindo SQL Injection) ou Statement (para queries estáticas sem parâmetros).

Por que PreparedStatement?

- ▲ Segurança: parâmetros são tratados como dados literais, nunca como código SQL.
- ▲ Se o nome de um cliente fosse ''; DROP TABLE clientes; --, uma concatenação simples executaria o DROP TABLE. Com PreparedStatement isso é impossível.
- ▲ Performance: o banco compila o SQL uma vez e reusa o plano de execução.

ClienteService.java Métodos Principais

Método	SQL Executado	Comportamento
registrarCliente(nome)	INSERT + RETURN_GENERATED_KEYS	Insere e retorna o objeto Cliente com ID gerado pelo banco.
listarTodos()	SELECT ORDER BY id	Retorna todos os clientes ordenados por ID de cadastro.
buscarPorId(id)	SELECT WHERE id = ?	Retorna Optional vazio se não encontrado.
buscarPorNome(nome)	SELECT WHERE nome = ? COLLATE NOCASE LIMIT 1	Usado no login: busca case-insensitive pelo nome.
remover(id)	DELETE WHERE id = ?	Remove e retorna boolean indicando sucesso.

PratoService.java - Seed Automático

Uma característica especial do PratoService é a semeadura automática do cardápio na primeira execução. No construtor, o service verifica via Database.tabelaVazia("pratos") se a tabela não possui registros e, em caso positivo, insere os 8 pratos padrão:

JAVA

```
public PratoService() {
    this.conn = Database.getInstance().getConn();
    if (db.tabelaVazia("pratos"))
        semearCardapioPadrao(); }

private void semearCardapioPadrao()
{ adicionarPrato("Frango Grelhado", 32.50, 15);
  adicionarPrato("File de Tilapia", 38.00, 20);
  adicionarPrato("Macarrao ao Molho", 27.00, 12);
  adicionarPrato("Picanha na Brasa", 65.00, 25);
  adicionarPrato("Salada Tropical", 18.00, 8);
  adicionarPrato("Suco Natural", 9.50, 3);
  adicionarPrato("Refrigerante", 7.00, 2);
  adicionarPrato("Agua Mineral", 4.00, 1);
}
```

PedidoService.java - Ciclo de Vida do Pedido

Método	SQL / Operação	Comportamento
abrirPedido(idCliente)	INSERT pedidos	Cria pedido com inicio=now(), fim=NULL, entregue=0
adicionarPrato(idPedido, idPrato)	INSERT pedido_pratos	Calcula posição (COUNT+1) e insere o item. Verifica se pedido está em aberto.
fecharPedido(idPedido)	UPDATE SET fim = now() WHERE fim IS NULL	Registra hora de encerramento. 'AND fim IS NULL' previne duplo fechamento.
marcarEntregue(idPedido)	UPDATE SET entregue = 1	Marca pagamento confirmado.
buscarPorId(id)	SELECT + carregarPratos()	Carrega cabeçalho do pedido e, separadamente, a lista de pratos via pedido_pratos.
listarAbertos()	SELECT WHERE fim IS NULL	Retoma apenas pedidos sem hora de encerramento.
listarPorCliente(idCliente)	SELECT WHERE id_cliente = ?	Histórico completo de um cliente específico.

AvaliacaoService.java

Método	SQL	Observação
registrar(idCliente, nome, nota, comentario)	INSERT avaliacoes	Valida nota (0-5), insere e retorna objeto Avaliacao.
listarTodas()	SELECT ORDER BY id DESC	Lista todas as avaliações, mais recentes primeiro.
calcularMedia()	SELECT AVG(nota) FROM avaliacoes	Retorna OptionalDouble (vazio se não há avaliações).

3.6 TUI.java - Interface com o Usu rio

O TUI   a maior classe do projeto, respons vel por toda a intera  o com o usu rio. Segue rigorosamente a regra de n o acessar o banco diretamente e de n o conter l gica de neg cio apenas delega aos services e exibe resultados.

Estado da Sess o

JAVA

```
private Cliente clienteAtual = null; // null = ningu m logado
private Pedido pedidoAtual = null; // null = sem pedido ativo
```

Esses dois campos de inst ncia representam o estado da sess o atual. Quando o usu rio faz login, ambos recebem valor. Quando sai (op  o 0 no menuRestaurante) ou encerra o pedido, ambos voltam a null.

Hierarquia de M todos

M�todo	Visib.	Responsabilidade
menuPrincipal()	public	Loop principal do programa. N�o encerra at� op��o 0 ser selecionada.
menuEntrar()	private	Login/Cadastro unificado. Chama clienteService.buscarPorNome() e decide entre login e cadastro.
exibirResumoCliente()	private	Exibe perfil completo: nome, ID, data de cadastro e hist�rico de pedidos com status.
menuRestaurante()	private	Menu principal do cliente logado. Ativo enquanto clienteAtual != null.
menuCardapio()	private	Loop de exib��o do card�pio. Aceita ID de prato at� o usu�rio digitar Q
verPedidoAtual()	private	Recarrega o pedido do banco e exibe itens com total calculado.
encerrarPedido()	private	Confirma pagamento, chama fecharPedido() e marcarEntregue(), zera a sess�o.
deixarAvaliacao()	private	Coleta nota (0-5 validada por lerNotaV�lida()) e coment�rio opcional.
menuAvaliacoes()	private	Exibe m�dia atual em estrelas e lista todas as avalia�es. N�o requer login.
msgSobreNos()	public	Exibe informa�es sobre o sistema e equipe de desenvolvimento.
cabecalho()	private	Imprime o banner ASCII art do sistema na inicializa��o.
aguardarEnter()	private	Pausa a execu��o at� o usu�rio pressionar Enter.
lerNotaV�lida()	private	Loop que garante que a nota digitada est� no intervalo [0, 5].

id	INTEGER	PRIMARY KEY AUTOINCREMENT	
id_cliente	INTEGER	NOT NULL, FK clientes(id)	Vínculo com o cliente dono do pedido.
inicio	TEXT	NOT NULL	ISO datetime de abertura. Ex: 2026-06-08T14:30:00.123456
fim	TEXT	NULL (opcional)	ISO datetime de encerramento. NULL = pedido em aberto.
entregue	INTEGER	NOT NULL, DEFAULT 0	0 = não pago; 1 = pago/encerrado. SQLite não tem tipo BOOLEAN.

Tabela: pedido_pratos

Coluna	Tipo SQL	Restrição	Descrição
id_pedido	INTEGER	NOT NULL, FK pedidos(id)	Referência ao pedido.
id_prato	INTEGER	NOT NULL, FK pratos(id)	Referência ao prato escolhido.
posicao	INTEGER	NOT NULL	Ordem de inserção do prato no pedido. Garante ORDER BY determinístico.

Tabela: avaliacoes

Coluna	Tipo SQL	Restrição	Descrição
id	INTEGER	PRIMARY KEY AUTOINCREMENT	
id_cliente	INTEGER	NOT NULL, FK clientes(id)	Referência ao cliente avaliador.
nome_cliente	TEXT	NOT NULL	Nome desnormalizado: mantido mesmo se o cliente for removido.
nota	INTEGER	NOT NULL	Valor inteiro de 0 a 5 (validado na camada TUI).
comentario	TEXT	DEFAULT ""	Comentário opcional. Nunca NULL - usa string vazia.
data	TEXT	NOT NULL	ISO datetime do momento da avaliação.

4.3 Relacionamentos e Chaves Estrangeiras

A tabela `pedido_pratos` é uma **tabela de junção** (junction table), solução canônica em SQL para representar relações Muitos-para-Muitos (N:N). Um pedido pode conter vários pratos, e um mesmo prato pode estar em vários pedidos.

A coluna `posicao` em `pedido_pratos` resolve o problema de ordenação: sem ela, a ordem de retorno dos pratos de um pedido seria indeterminada (comportamento não-garantido pelo SQLite sem ORDER BY).

Com a posição registrada, a query `SELECT id_prato ... ORDER BY posicao` sempre retorna os pratos na mesma sequência em que foram adicionados pelo cliente.

5 Fluxos de Navegação e Casos de Uso

5.1 Fluxo Completo do Sistema

#	Etapa	Descrição Técnica
1	Inicialização	Main.main() instancia TUI. TUI instancia os 4 services. Database.getInstance() abre o arquivo SQLite, executa PRAGMA e cria as 5 tabelas. PratoService verifica se pratos está vazio e semeia o cardápio.
2	Menu Principal	Loop exibe as opções: Entrar (1), Ver Avaliações (2), Sobre Nós (3), Sair (0). Permanece ativo até opção 0.
3	Acesso (Opção 1)	menuEntrar(): usuário digita o nome. ClienteService.buscarPorNome() executa SELECT com COLLATE NOCASE.
4a	Login (nome existe)	clienteAtual recebe o cliente encontrado. exibirResumoCliente() mostra perfil. PedidoService.verificaPedidoAberto() se houver, retoma; se não, abre novo.
4b	Cadastro (nome novo)	Oferece opção 1 = Cadastrar. ClienteService.registrarCliente() executa INSERT. PedidoService.abrirPedido() abre pedido imediatamente.
5	Menu Restaurante	Loop com 4 opções: Cardápio (1), Meu Pedido (2), Pagar (3), Avaliar (4), Sair (0).
6	Cardápio	menuCardapio(): exibe todos os pratos. Usuário digita ID para adicionar. PedidoService.adicionarPrato() insere em pedido_pratos com posição calculada.
7	Encerramento	encerrarPedido(): mostra resumo com total. Confirma. PedidoService.fecharPedido() registra hora; marcarEntregue() seta entregue=1. clienteAtual e pedidoAtual voltam a null.
8	Avaliação	Disponível logado (opção 4 no menuRestaurante) ou sem login via menu principal (opção 2). AvaliacaoService.registrar() insere na tabela avaliacoes.

5.2 Caso de Uso: Login e Cadastro Unificado

O sistema não possui telas separadas de login e cadastro. O fluxo unificado pelo nome do cliente é uma decisão de UX deliberada para reduzir a fricção de acesso ao sistema:

```
// TUI.menuEntrar() – lógica de decisão
Optional<Cliente> encontrado = clienteService.buscarPorNome(nome);

if (encontrado.isPresent()) {
// LOGIN: cliente existe → carrega perfil + retoma ou abre pedido
clienteAtual = encontrado.get();
exibirResumoCliente();
List<Pedido> abertos = pedidoService.listarPorCliente(id)
    .stream().filter(p -> p.getFim() == null).toList();
pedidoAtual = abertos.isEmpty()
    ? pedidoService.abrirPedido(id) // abre novo
    : abertos.get(0); // retoma existente
} else {
// CADASTRO: nome não encontrado → oferece registro
// Se confirmado: registrarCliente() + abrirPedido()
}
}
```

5.3 Caso de Uso: Realizar Pedido

O ciclo completo de um pedido envolve três operações em duas tabelas distintas:

- Abertura: INSERT INTO pedidos com fim = NULL e entregue = 0.
- Adição de itens: para cada prato, INSERT INTO pedido_pratos com a posição calculada por SELECT COUNT(*) FROM pedido_pratos WHERE id_pedido = ?.
- Encerramento: UPDATE pedidos SET fim = ? WHERE id = ? AND fim IS NULL seguido de UPDATE pedidos SET entregue = 1 WHERE id = ?.

A cláusula AND fim IS NULL no UPDATE de fechamento é uma salvaguarda idempotente: se por algum motivo o método fecharPedido() for chamado duas vezes no mesmo pedido, a segunda chamada não terá efeito e retornará false (0 linhas afetadas).

5.4 Caso de Uso: Sistema de Avaliações

As avaliações possuem dois pontos de acesso com comportamentos distintos:

Contexto	Acesso	Comportamento
Dentro da sessão (logado)	Opção 4 no menuRestaurante	O nome do cliente é preenchido automaticamente a partir de clienteAtual.getNome(). Não é necessário digitá-lo.
Sem login (público)	Opção 2 no menuPrincipal	Qualquer pessoa pode VER as avaliações e a média. Para deixar uma avaliação, é necessário estar logado.

A média é calculada diretamente no banco via SELECT AVG(nota) FROM avaliacoes, sem nenhum loop de soma em Java. O uso de OptionalDouble como retorno trata de forma elegante o caso de não haver nenhuma avaliação registrada.

6 Levantamento de Requisitos

6.1 Requisitos Funcionais

Os requisitos funcionais descrevem as capacidades operacionais do sistema o que ele deve ser capaz de fazer:

ID	Requisito	Descrição	Implementação
RF01	Cadastro de Cliente	O sistema deve permitir o registro de novos clientes pelo nome, com armazenamento da data de entrada. Após o cadastro, um pedido deve ser aberto automaticamente.	ClienteService.registrarCliente()
RF02	Login por Nome	O sistema deve identificar clientes existentes pelo nome de forma case-insensitive (sem distinção de maiúsculas/minúsculas), sem necessidade de senha.	ClienteService.buscarPorNome()
RF03	Exibição de Perfil	Ao fazer login, o cliente deve visualizar seu nome, ID, data de cadastro, número total de pedidos e histórico dos últimos 5 pedidos com status.	TUI.exibirResumoCliente()
RF04	Retomada de Pedido	Se o cliente possuir um pedido em aberto ao fazer login, o sistema deve retomá-lo automaticamente em vez de abrir um novo.	TUI.menuEntrar()
RF05	Listagem do Cardápio	O sistema deve exibir o cardápio completo com ID, nome, preço e tempo de preparo de cada prato, formatados em colunas alinhadas.	PratoService.listarTodos()
RF06	Adição de Pratos ao Pedido	O cliente deve poder adicionar quantos pratos desejar ao pedido ativo, informando o ID do prato. O sistema deve validar se o ID existe.	PedidoService.adicionarPrato()
RF07	Visualização do Pedido	O cliente deve poder visualizar a qualquer momento os pratos do pedido atual com o total calculado em reais.	TUI.verPedidoAtual()
RF08	Encerramento e Pagamento	O sistema deve registrar a hora de encerramento do pedido e marcar como pago/entregue após confirmação do cliente.	PedidoService.fecharPedido()
RF09	Registro de Avaliação	Clientes logados devem poder registrar uma avaliação com nota de 0 a 5 e comentário opcional. A nota deve ser validada antes do registro.	AvaliacaoService.registrar()
RF10	Listagem de Avaliações	Qualquer usuário (logado ou não) deve poder visualizar todas as avaliações registradas com nome do cliente, nota em estrelas e data.	AvaliacaoService.listarTodas()
RF11	Média das Avaliações	O sistema deve calcular e exibir a média geral das avaliações em formato de estrelas (★) com uma casa decimal.	AvaliacaoService.calcularMedia()

RF12	Persistência Total	Todos os dados (clientes, pratos, pedidos, itens, avaliações) devem ser persistidos em banco de dados e mantidos entre execuções do programa.	Database + todos os Services
------	--------------------	---	------------------------------

6.2 Requisitos Não Funcionais

Os requisitos não funcionais definem as restrições de qualidade, tecnológicas e de processo impostas ao sistema:

ID	Requisito	Descrição	Status
RNF01	Linguagem Java 21	O sistema deve ser desenvolvido integralmente em Java 21, aproveitando recursos modernos da linguagem: text blocks ("""..."""), switch expressions (->), Stream API, Optional, List.copyOf() e LocalDateTime.	Confirmado
RNF02	Execução via Terminal	Toda a interface deve ser operada exclusivamente no terminal de linha de comando (TUI). Nenhuma janela gráfica (Swing, JavaFX ou AWT) deve ser utilizada.	Confirmado
RNF03	Banco de Dados SQL	A persistência de dados deve utilizar um banco de dados relacional SQL. Foi escolhido SQLite por ser embutido, sem servidor, e adequado para aplicações locais.	Confirmado - sqlite-jdbc 3.47.1.0
RNF04	Segurança de Queries	Toda query que receba parâmetros externos (IDs, nomes, textos) deve utilizar PreparedStatement, impedindo ataques de SQL Injection.	Confirmado
RNF05	Código Organizado	O código deve seguir arquitetura em camadas (TUI / Services / Models / Database) com responsabilidades claramente delimitadas entre as classes.	Confirmado - 4 camadas
RNF06	Independência de Rede	O sistema deve funcionar completamente offline. Nenhuma operação deve depender de conexão com a internet.	Confirmado - SQLite local
RNF07	Validação de Entradas	Todas as entradas do usuário devem ser validadas: campos numéricos rejeitam texto (NumberFormatException tratada), nota de avaliação deve estar no intervalo [0,5], campos obrigatórios não podem ser vazios.	Confirmado - Utils.java
RNF08	Build via Gradle	O projeto deve ser construído e gerenciado pelo Gradle com wrapper incluso no repositório, permitindo build com ./gradlew build sem pré-instalação do Gradle no sistema.	Confirmado - Gradle 9.3
RNF09	Fat-Jar Executável	O artefato final de distribuição (jar) deve incluir todas as dependências (sqlite-jdbc) para execução standalone com java -jar sem configuração adicional de classpath.	Configurado no build.gradle
RNF10	Uso de Git/GitHub	O código-fonte deve ser versionado com Git e hospedado no GitHub. O histórico de commits deve refletir o progresso incremental do desenvolvimento.	github.com/Meinards15/trabalho-algoritmo

7 Considerações Finais

O sistema RestaurARTE demonstra na prática a aplicação dos principais conceitos estudados na disciplina de Algoritmos e Programação: orientação a objetos com separação de responsabilidades, persistência de dados com SQL e JDBC, estruturas de dados como listas e opcionais, e padrões de projeto como Singleton e Service Layer.

A decisão de migrar a persistência de arquivos de texto simples para um banco de dados SQLite trouxe ganhos concretos e mensuráveis: busca por nome com complexidade $O(\log n)$ em vez de $O(n)$, integridade referencial garantida por chaves estrangeiras, operações atômicas que eliminam a possibilidade de corrupção de dados por interrupção, e queries declarativas que simplificam operações como cálculo de média (AVG) e filtragem de pedidos abertos (WHERE fim IS NULL).

O sistema foi projetado com extensibilidade em mente. Para adicionar uma nova funcionalidade, o padrão a seguir é consistente: criar o modelo correspondente, criar o service com os métodos CRUD necessários, adicionar a tabela no DDL de Database.criarEsquema() e expor os menus correspondentes no TUI. A arquitetura em camadas garante que nenhuma dessas adições impacte as camadas não relacionadas.

Como evolução natural do projeto, o sistema pode ser estendido com: sistema de categorias no cardápio, controle de mesas do restaurante, relatórios de vendas por período, exportação de dados, ou migração para um banco de dados com suporte a múltiplos usuários simultâneos (PostgreSQL ou MySQL) com alteração mínima no código – apenas a string de conexão JDBC e o driver precisariam ser substituídos.